universität innsbruck

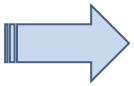
Einführung

Einführung in die Programmierung

Michael Tschuggnall
Universität Innsbruck

Überblick

- Einführung
- Vorschau
- Variablen und Datentypen
- Operatoren und Ausdrücke
- Kontrollstrukturen



- Programmierung allgemein
- C-Hintergrund
- Das erste C-Programm

- Funktionen
- Arrays
- Zeiger
- Strukturen, Unions, Bitfelder
- Speicherklassen
- Dynamische Speicherverwaltung

- Präprozessor
- Modulare Programmierung
- Ein- und Ausgabe
- gcc und make

Programmierung allgemein

Maschinensprache

- Ein Computer ist eine Maschine, die ein Problem löst, indem sie Befehle (instructions) ausführt.
- Eine Central Processing Unit (CPU) „kennt“ nur eine begrenzte Menge einfacher Maschinenbefehle, die als Maschinensprache (machine language) bezeichnet wird.
  - Unterschiedliche CPUs können unterschiedliche Maschinensprachen haben!
- Eine Folge von Befehlen, welche zur Ausführung einer bestimmten Aufgabe zusammengestellt wurde, wird als Programm bezeichnet.
- **Alle Programme** müssen in Maschinensprache umgewandelt werden, ehe sie ausführbar sind.
- Beispiel (ARM64 Architektur, Befehl als Bitmuster):
  - `00010001000000000000010000000000`
    - Den Wert im Register `w0` um `1` erhöhen.
    - `add w0, w0, #1`

Höhere Programmiersprachen

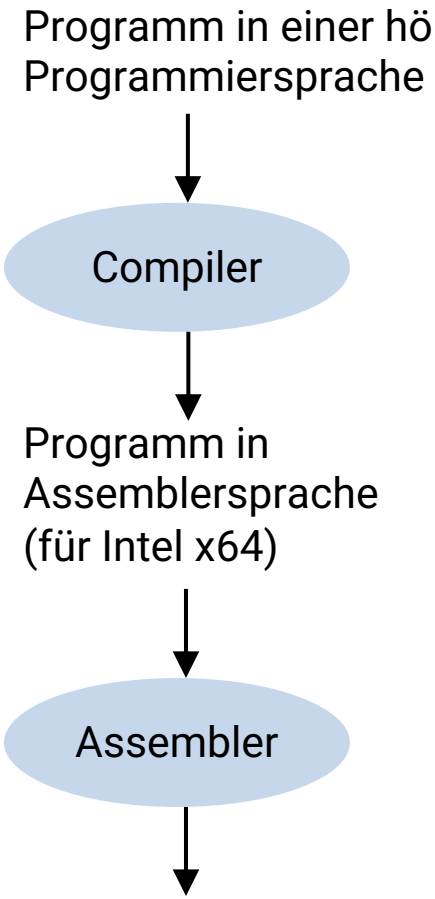
- In einem Maschinenprogramm muss dem Prozessor jeder auszuführende Befehl in Maschinensprache vorgeschrieben werden.
  - Sehr zeitaufwändig und fehleranfällig!
- Menschen können Maschinensprachen nur für sehr kleine Programme benutzen und daher wurden **höhere Programmiersprachen** entwickelt.
  - Diese sollten es ermöglichen, eine Problemlösung in einer eher fachspezifischen Notation anzugeben.
  - Ein Programm muss dadurch nicht mehr in CPU-spezifischer Maschinensprache geschrieben werden.
    - Abstraktion von der Hardware.
- Da die Maschine wiederum die Befehle der höheren Programmiersprache nicht kennt, müssen diese in die Maschinensprache **übersetzt** werden.

Compilieren

- Beim Übersetzen eines in einer höheren Programmiersprache geschriebenen Programms in eine ausführbare Datei wird jeder Befehl in eine entsprechende **Folge von Maschinenbefehlen** übersetzt und in einer Datei gespeichert.
- Ausgeführt wird dann das Maschinenprogramm aus dieser ausführbaren Datei.
- Das Programm, das für die Übersetzung zuständig ist, wird allgemein **Compiler** genannt.
  - In der Realität setzt sich der Übersetzungsvorgang aus mehreren Stufen zusammen, und es werden noch zusätzliche Programme benötigt.

Beispiel – C und Intel-Prozessor



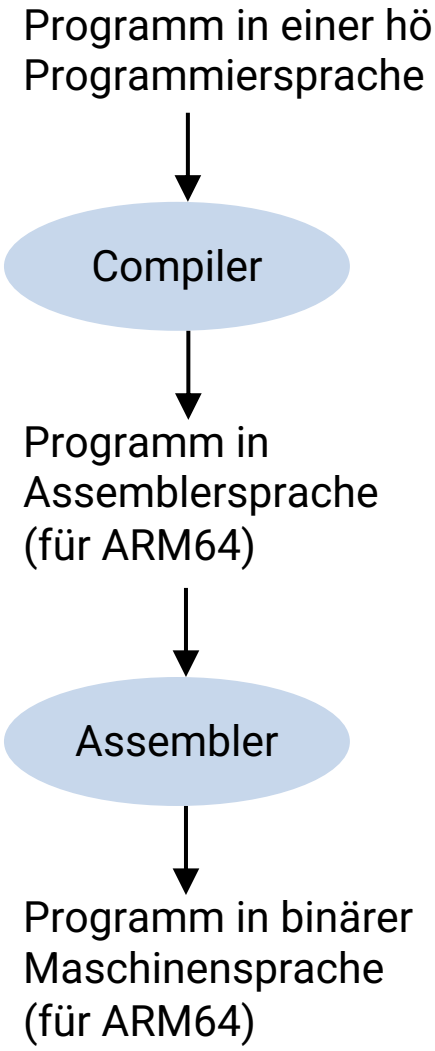



```c
void swap(int v[], int k) {
    int tmp = v[k];
    v[k] = v[k + 1];
    v[k + 1] = tmp;
}
```

```
swap:
    movsxd rsi,esi
    shl    rsi,0x2
    lea    rdx,[rdi+rsi*1]
    mov    ecx,DWORD PTR [rdx]
    lea    rax,[rdi+rsi*1+0x4]
    mov    esi,DWORD PTR [rax]
    mov    DWORD PTR [rdx],esi
    mov    DWORD PTR [rax],ecx
    ret
```

```
01001000011000111111011001001000
11000001111001100000001001001000
10001101000101000011011110001011
00001010010010001000110101000100
00110111000001001000101100110000
10001001001100101000100100001000
11000011
```

Beispiel – C und ARM-Prozessor




```c
void swap(int v[], int k) {
    int tmp = v[k];
    v[k] = v[k + 1];
    v[k + 1] = tmp;
}
```

```
swap:
    sxtw  x2, w1
    sbfiz x1, x1, #2, #32
    ldr   w3, [x0, x2, lsl #2]
    add   x1, x1, #0x4
    ldr   w4, [x0, x1]
    str   w4, [x0, x2, lsl #2]
    str   w3, [x0, x1]
    ret
```

```
10010011010000001111110000100010
10010011011111100111110000100001
10111000011000100111100000000011
10010001000000000001000000100001
10111000011000010110100000000100
10111000001000100111100000000100
10111000001000010110100000000011
11010110010111110000001111000000
```

Interpretieren

- Bei dieser Übersetzungsart werden nicht alle Befehle des in der höheren Sprache geschriebenen Programms auf einmal in ein Maschinenprogramm übersetzt und abgespeichert.
- Stattdessen werden die Befehle bei der Ausführung einzeln nacheinander in entsprechende Folgen von Maschinenbefehlen übersetzt und diese dann ausgeführt.
- Das Programm, das für die Übersetzung und Ausführung der einzelnen Befehle zuständig ist, wird Interpreter genannt.

C-Hintergrund

Eigenschaften von C



- Imperative Programmiersprache
- Wenige Schlüsselwörter (reservierte Wörter)
- Direkte Speicherzugriffe und sehr hardwarenahe Programmierung
- Schwach ausgeprägtes Modulkonzept
- Trotz weniger Schlüsselwörter sehr mächtige Sprache
  - **Vor- und Nachteil!**

Wie hat sich C entwickelt?



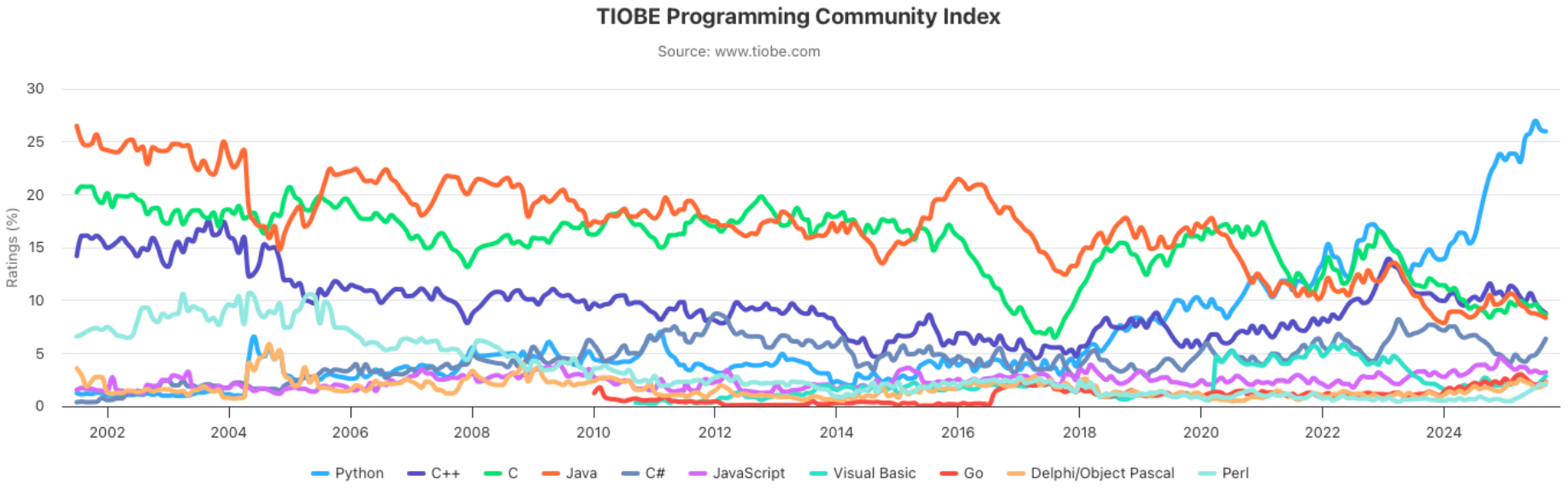
- Woher stammt C?
  - AT&T Bell Laboratories
  - 1969 erste C Version von Dennis Ritchie und Ken Thompson für Unix entwickelt
- Versionen
  - 1978 – *K&R*-C (Brian W. Kernighan und Dennis Ritchie) in der ersten Auflage des Buches *The C Programming Language*.
  - 1989 – C89 Standard (ANSI C, Standard C)
  - 1990 – ANSI C von ISO übernommen (C 90)
  - 1999 – C99 Standard
  - 2011 – C11 Standard
  - **2018 – C18 Standard (auch C17 genannt)**
  - 2023 – C23 Standard
- In dieser Lehrveranstaltung (VO+UE) verwenden wir den C18-Standard.

Aktualität von C



- Wo wird C heute verwendet?
  - Betriebssysteme (z.B. Linux) und Systemprogrammierung
  - Hardwarenahe Programmierung
  - Programmierung eingebetteter Systeme
  - Spieleprogrammierung
  - Interpreter für Programmiersprachen (Ruby, Python etc.)
  - ….
- C ist die Grundlage weiterer Programmiersprachen.
  - C++ - Objektorientierte Variante von C.
  - Objective C – Objektorientierte Erweiterung von C.
  - Weitere Sprachen, die von C beeinflusst wurden:
    - Java, C#, Swift, Python, R, Rust …
- C-Compiler sind auf fast allen Systemen verfügbar.

TIOBE-Index



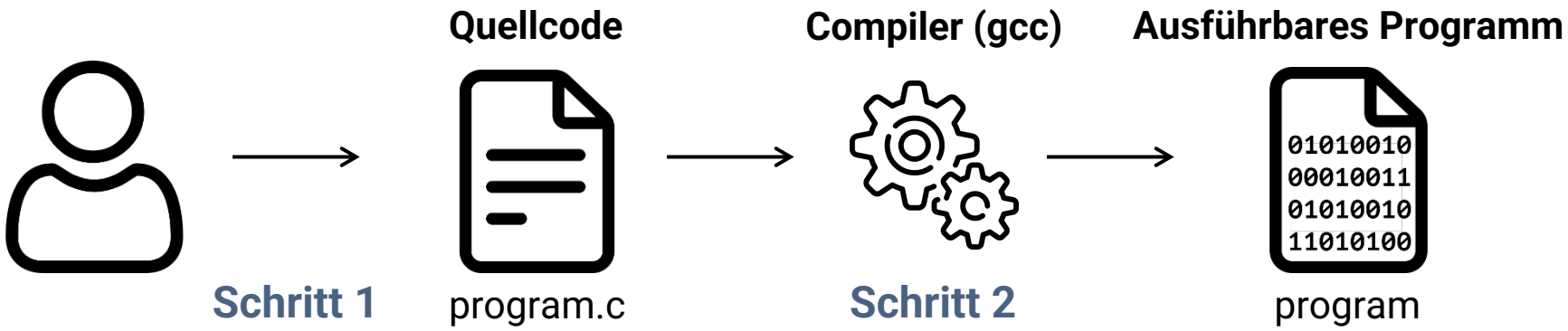
Stand: September 2026

Das erste C-Programm

C-Programmierung

- Grundsätzlich gibt es zwei Arten von Dateien.
  - Quelltextdateien:
    - Enthalten den Quellcode.
    - Typischerweise wird als Dateiendung `.c` verwendet!
  - Headerdateien:
    - Beinhalten in der Regel Informationen, welche für die Übersetzung benötigt werden.
    - Beinhalten Informationen über neu definierte Datentypen.
    - Typischerweise wird als Dateiendung `.h` verwendet!
- Kleine Programme können in einer einzigen Quelltextdatei geschrieben werden.

Einfache Programmerzeugung (eine Quellcodedatei)



- Schritt 1:
  - Erstellen des Programms in einem beliebigen Texteditor (z.B. kate, gedit, nano) oder einer IDE (z.B. Eclipse, CLion, Visual Studio Code),
  - Abspeichern unter einem beliebigen Namen:
    - Der Name sollte aber Sinn ergeben und auf **`.c`** enden (z.B. **`program.c`**).
- Schritt 2 (sehr vereinfacht!):
  - Kompilieren des Programms.
  - Beispiel für das Kompilieren auf der Kommandozeile eines ZID-Rechners:

    `gcc program.c -o program` (oder **`cc program.c -o program`**)
  - Erzeugt das ausführbare Programm **`program`**.

gcc

- Compiler (und noch mehr!)
  - Für unterschiedliche Betriebssysteme und Architekturen erhältlich.
  - In den meisten Linux-Distributionen enthalten.
- Minimaler Aufruf:

```
gcc program.c
```

  - Erzeugt ausführbares Programm **`a.out`** (assembler output) unter Linux.
  - Unter Windows: **`a.exe`** (z.B. von MinGW erzeugt).
- Aufruf mit Output-Datei:

```
gcc program.c -o program
```

- Aufruf mit mehreren Flags:
  - Mehrere Dateien, Optimierung etc.
  - Aufrufkonvention für diese Vorlesung und die Übung:

```
gcc -Wall -Wextra -Werror -Wpedantic -std=c18 program.c -o program
```

Das erste C-Programm

hello_world.c

```
1  /* Das erste C-Programm */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      printf("Hello World!\n");
7      return EXIT_SUCCESS;
8  }
```



Linux Kommandozeile (Quellcode-Datei hat den Namen hello_world.c):

```
$ gcc -Wall -Wextra -Werror -Wpedantic -std=c18 hello_world.c -o hello_world
$ ./hello_world
Hello World!
```

Das erste C-Programm (Erklärung zu 1.)

hello_world.c

```c
/* Das erste C-Programm */
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("Hello World!\n");
    return EXIT_SUCCESS;
}
```

1.

- Hier steht ein Kommentar.
- Damit wird der Programmcode kommentiert!
- Der Inhalt eines Kommentars hat keinen Einfluss auf den Ablauf des Programms.
- Es existieren zwei Formen:
  - Blockkommentare: `/* … */` Einzeilig oder mehrzeilig
  - Zeilenkommentare: `//…` Einzeilig

Das erste C-Programm (Erklärung zu 2.)

hello_world.c

```c
/* Das erste C-Programm */
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("Hello World!\n");
    return EXIT_SUCCESS;
}
```



- Präprozessordirektiven (beginnen mit **#**)
- Diese Direktiven werden vom Präprozessor verarbeitet.
  - Der Präprozessor bearbeitet das gegebene Programm (durch reine Textersetzung) und übergibt das Ergebnis an den eigentlichen Compiler.
- Die **`#include`**-Direktive dient zum Einbinden von Dateien.
  - Der Inhalt dieser Dateien wird vom Präprozessor an dieser Stelle eingesetzt!
- In diesem Fall wird zum Beispiel die Datei **`stdio.h`** eingebunden, die Informationen über den Befehl **`printf`** beinhaltet.

Das erste C-Programm (Erklärung zu 3.)

hello_world.c

```c
/* Das erste C-Programm */
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("Hello World!\n");
    return EXIT_SUCCESS;
}
```



- Hier beginnt das eigentliche Programm.
- **`main()`** ist eine Funktion, die in jedem Programm enthalten sein muss.
  - Ein Programm kann aus mehreren Funktionen bestehen.
- Das ist der Einstiegspunkt für jedes C-Programm.
- In diesem Fall werden keine Daten übergeben, daher **`(void)`** und es wird etwas zurückgegeben (vom Datentyp **`int`** – wird noch ausführlich erklärt).
- Die Anweisungen des Programms stehen zwischen **`{`** und **`}`**.

Das erste C-Programm (Erklärung zu 4.)

hello_world.c

```c
/* Das erste C-Programm */
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("Hello World!\n");
    return EXIT_SUCCESS;
}
```

4.

- An dieser Stelle wird die Funktion **`printf`** aufgerufen und die Zeichenkette **`Hello World!`** übergeben (alle Zeichen zwischen den Anführungszeichen).
  - Am Ende wird ein `\n` hinzugefügt. Damit wird ein Zeilenumbruch erzwungen.
- Dieser Aufruf wird mit einem Semikolon abgeschlossen!
- Resultat bei Ausführung: Die Zeichenkette wird auf der Standardausgabe (Konsole) ausgegeben.

Das erste C-Programm (Erklärung zu 5.)

hello_world.c

```c
/* Das erste C-Programm */
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("Hello World!\n");
    return EXIT_SUCCESS;
}
```



5.

- Rückgabe eines Wertes (wird noch ausführlich erklärt).
- Hier könnte 0 (**`return 0;`**) zurückgegeben werden.
  - `EXIT_SUCCESS` macht den Code lesbarer.
  - Ein Programm kann bei Bedarf durch `return EXIT_FAILURE;` nicht erfolgreich beendet werden.
  - Die Konstanten `EXIT_SUCCESS` und `EXIT_FAILURE` sind in der Headerdatei `stdlib.h` definiert.

Aufgabe

- Das folgende C-Programm kompiliert nicht erfolgreich. Was ist der Fehler?

```c
int main(void) {
    printf("Was ist hier falsch?\n");
    printf("Es fehlt was!\n");
    return EXIT_SUCCESS;
}
```

Aufgabe - Lösung

- Das folgende C-Programm kompiliert nicht erfolgreich. Was ist der Fehler?

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("Was ist hier falsch?\n");
    printf("Es fehlt was!\n");
    return EXIT_SUCCESS;
}
```

Welche Zeichen dürfen in C verwendet werden?

- Mindestanforderung lt. C-Standard:
  - Buchstaben
    - `A B C D E F G H I J K L M N O P Q R S T U V W X Y Z`
    - `a b c d e f g h i j k l m n o p q r s t u v w x y z`
  - Dezimalziffern
    - `0 1 2 3 4 5 6 7 8 9`
  - 29 Grafiksymbole
    - `! " % & / ( ) [ ] { } \ ? = ' # + * ~ - _ . : ; , | < > ^`
  - Whitespace-Zeichen
    - Leerzeichen, Tabulator (horizontal, vertikal), neue Zeile, neue Seite

- Die meisten modernen Compiler unterstützen aber auch erweiterte Zeichensätze (ISO 8859-1, UTF-8 usw.)
  - Umlaute
  - Emojis
  - ...